

Lab Report – Logical Reasoning for Planning

Claude (AI assistant, Anthropic)

Task 0: Planning problem specification

(a) $I = \{At(Robot, A), At(Package, A)\}$. (b) $G = \{At(Package, C)\}$. (c,d) Actions:

Action	Preconditions	Effects
Move(x, y), (x, y) $\in \{(A, B), (B, A), (B, C), (C, B)\}$	At(Robot, x)	$\neg At(Robot, x)$, At(Robot, y)
PickUp(Package, l)	At(Robot, l), At(Package, l)	$\neg At(Package, l)$, Holding(Package)
Drop(Package, l)	At(Robot, l), Holding(Package)	$\neg Holding(Package)$, At(Package, l)

Initially applicable? PickUp(Package,A): yes, both preconditions At(Robot,A) and At(Package,A) are in I . Drop(Package,C): no, At(Robot,C) and Holding(Package) are both false in I . (The planner confirms that the only initially applicable actions are Move(A,B) and PickUp(Package,A).) An action in the list is applicable only when $S \models Pre(a)$.

Task 1: Manual plan

Note: the lab's suggested sequence Move(A,B), PickUp(Package,B), ... is not valid, since the package stays at A (PickUp at B needs At(Package,B)). Valid 4-step plan:

State	Facts
S_0	At(Robot,A), At(Package,A)
S_1 = after PickUp(Package,A)	At(Robot,A), Holding(Package)
S_2 = after Move(A,B)	At(Robot,B), Holding(Package)
S_3 = after Move(B,C)	At(Robot,C), Holding(Package)
S_4 = after Drop(Package,C)	At(Robot,C), At(Package,C) $\models G$

Task 2: Prompt and generated program

The prompt (the lab's suggested prompt plus the warehouse definition) is in the appendix and `prompts.txt`. I (Claude) wrote `planner.py` and used it without modification. Where the ideas appear: *preconditions* $\rightarrow$ `applicable()` (`pos_pre` $\subseteq S$ and `neg_pre` $\cap S = \emptyset$); *effects* $\rightarrow$ `apply_action()` = $(S \setminus \text{neg_eff}) \cup \text{pos_eff}$; *goal* $\rightarrow$ `set(goal)` $\leq s$ when a state is dequeued, where planning stops; *BFS* $\rightarrow$ the `deque` frontier with a `parent` dictionary, exploring all applicable actions level by level. Assumptions stated: closed-world assumption, only the package is movable, Move only along the listed edges.

Task 3: Tests

Test	Initial / goal	Plan found?	Plan	Valid?
A solvable	I / At(Package,C)	yes	PickUp(Package,A), Move(A,B), Move(B,C), Drop(Package,C)	yes (re-executed, each pre-condition checked) n/a
B no PickUp	I / At(Package,C)	No plan found	–	n/a
C1 extra Move(A,C), goal package at C	I / At(Package,C)	yes	PickUp(Package,A), Move(A,C), Drop(Package,C)	yes
C2 extra Move(A,C), goal robot at C	I / At(Robot,C)	yes	Move(A,C)	yes; package still at A
C3 extra Move(A,C), no PickUp	I / At(Package,C)	No plan found	–	n/a

Test B: the planner reports failure instead of inventing an action. Test C: when the robot alone can reach C, the package goal is still not satisfied (C3) and in C2 the robot reaches C without the package, so the planner does not conflate the two. An invalid plan (Move(A,B), Move(B,C), Drop) is rejected by the verifier (robot never held the package).

Task 4: Logic and Search

Flow: Current state $\rightarrow$ check preconditions $\rightarrow$ **apply effects (remove negative, add positive)** $\rightarrow$ generate successor state $\rightarrow$ search over alternatives $\rightarrow$ goal? Logic (entailment $S \models \text{Pre}(a)$ and the effect rules) decides which actions are possible and what the next state is; search (BFS with a visited set) decides in which order the applicable actions are tried and stops at the first state containing the goal. Logic determines what is possible; search determines what to try.

Task 5 (optional): LLM explanation versus independent verification

As the LLM, I produced the justification below from prompt 3 in the appendix and compared it with the transitions my program executed; they matched: PickUp needs At(Robot,A), At(Package,A) which hold in S_0 ; Move(A,B) needs At(Robot,A), true in S_1 ; Move(B,C) needs At(Robot,B), true in S_2 ; Drop needs At(Robot,C), Holding(Package), true in S_3 . I trust (b) the independently executed state transitions more: they are computed mechanically from the action definitions, while an explanation is generated text that can sound right yet be wrong, for instance it could claim PickUp(Package,B) is valid as in the lab's example sequence. A generated explanation is not an independent verification.

Task 6–8: Prolog

Run with SWI-Prolog 10.0.2 on `planner.pl`:

```
?- can_move(a,b).    true
?- can_move(a,c).    false
?- valid_move(a,b).  true
?- valid_move(b,c).  true
?- valid_move(a,c).  false
?- reduce_speed.     true
?- valid_path([a,b,c]). true
?- valid_path([a,c]). false
```

Task 6. (a) `can_move(a,b)` is true because the fact `connected(a,b)` exists and the rule `can_move(X,Y) :- connected(X,Y)` derives it. (b) `can_move(a,c)` is not established because there is no `connected(a,c)` fact and no rule that derives it (Prolog failure means “not provable” under the closed-world assumption, not “proven false”). (c) The rule is the implication $\text{Connected}(X,Y) \rightarrow \text{CanMove}(X,Y)$ read right to left (head `:-` body), universally quantified over X,Y .

Task 7. `valid_move(a,b)` and `valid_move(b,c)` succeed, `valid_move(a,c)` fails. *Challenge:* the proposed `Move(a,c)` is not supported by the warehouse knowledge: no `connected(a,c)`, so Prolog rejects it, a check independent of the Python generator.

Task 8. `reduce_speed` succeeds: `wet_road` is a fact; `slippery` follows from it by the rule; `reduce_speed` follows from `slippery`. $\text{Wet_road} \Rightarrow (\text{wet_road} \rightarrow \text{slippery}) \Rightarrow (\text{slippery} \rightarrow \text{reduce_speed}) \Rightarrow \text{reduce_speed}$.

Think About It answers

Applicability requires all preconditions to hold, not just that the action is listed (PickUp at B is listed but inapplicable). Logic determines what is possible; search determines what to try. A generated explanation is not an independent verification. Prolog is not identical to classical logic: it uses unification, backtracking, clause order and negation as failure.

Reflection Questions

1. Specifying preconditions and effects first fixes the meaning of “correct” so the generated code can be checked against it, and stops the LLM from inventing its own action semantics.
2. Without precondition checks the robot could Drop a package it is not holding, or PickUp at B when the package is at A, producing a “plan” that is impossible in the real world; in Test C the robot could wrongly “deliver” the package by just reaching C.
3. A plan can look reasonable (Move, Move, Drop) yet skip PickUp; validity means every action’s preconditions hold in the state where it executes and the final state entails the goal.
4. The LLM wrote the state representation, `applicable/apply`, BFS and plan printing, plus the test harness and Prolog file, quickly.
5. What I verified independently: that the action definitions matched the specification, that re-executing the plan step by step succeeds, and the impossible, irrelevant-action and invalid-plan cases; I also noticed that the lab’s example sequence with PickUp at B is invalid.
6. In checking $S \models \text{Pre}(a)$, applying effects, the goal test $S \models G$, and in the Prolog facts and rules (modus ponens chaining).
7. Planning is search in the space of states: states are sets of propositions, actions are the transitions (with logic deciding applicability), and BFS is the same algorithm used for path search in the previous module.

Optional reflection (7.2)

(1) A fact is an unconditional statement (`connected(a,b).`); a rule derives a head from body conditions (`can_move(X,Y) :- connected(X,Y).`). (2) A query asks whether the goal is derivable from the facts and rules, i.e. whether it follows from the knowledge base. (3) Prolog checks the plan against an independent logical description, so a bug in the Python generator cannot validate itself. (4) An LLM-produced plan may be plausible but wrong; an independent verifier catches invalid actions without trusting the generator.

Appendix A: Prompts

Prompt 1 (Task 2: planner)

I want to implement a simple planning agent in Python.

Represent a state as a set of logical propositions.

Each action should contain: a name; positive preconditions; negative preconditions; positive effects; negative effects.

An action is applicable if all of its preconditions are satisfied by the current state.

When an action is applied: 1. remove its negative effects from the state; 2. add its positive effects to the state.

Use breadth-first search to find a sequence of actions that achieves a specified goal.

The program should also: detect when no plan exists; print the resulting sequence of actions; print the states reached after each action.

Explain the implementation and identify any assumptions you make.

Run the generated program on the warehouse problem.

(Warehouse: locations A, B, C; Move along A-B, B-A, B-C, C-B; PickUp(Package,L) needs At(Robot,L), At(Package,L), effects -At(Package,L), +Holdin

Prompt 2 (Task 3: tests)

Add tests: A) the original warehouse problem, verifying every action of the returned plan by re-executing it; B) remove PickUp and check the plan

Prompt 3 (Task 5: LLM explanation of validity)

For every action in the plan, identify its preconditions and show that those preconditions are satisfied in the state in which the action is executed

Prompt 4 (Tasks 6-8: Prolog)

Write `planner.pl` with the `connected/2` facts, `can_move/2` and `valid_move/2` rules, the `wet_road/slippery/reduce_speed` program, and a `valid_path/1` ru

Appendix B: Output

Python

```
Initially applicable actions: ['Move(A,B)', 'PickUp(Package,A)']
--- Test A: solvable warehouse problem
Initial: At(Package,A), At(Robot,A)
Goal   : At(Package,C)
Plan: PickUp(Package,A) -> Move(A,B) -> Move(B,C) -> Drop(Package,C)
      S0: At(Package,A), At(Robot,A)
      S1: At(Robot,A), Holding(Package)
      S2: At(Robot,B), Holding(Package)
      S3: At(Robot,C), Holding(Package)
      S4: At(Package,C), At(Robot,C)
Plan verified independently: True

--- Test B: no PickUp action
Initial: At(Package,A), At(Robot,A)
Goal   : At(Package,C)
No plan found

--- Test C1: goal = Package at C, with extra Move(A,C) shortcut
Initial: At(Package,A), At(Robot,A)
Goal   : At(Package,C)
Plan: PickUp(Package,A) -> Move(A,C) -> Drop(Package,C)
      S0: At(Package,A), At(Robot,A)
      S1: At(Robot,A), Holding(Package)
      S2: At(Robot,C), Holding(Package)
      S3: At(Package,C), At(Robot,C)
Plan verified independently: True

--- Test C2: goal = Robot at C (package need not move)
Initial: At(Package,A), At(Robot,A)
Goal   : At(Robot,C)
Plan: Move(A,C)
      S0: At(Package,A), At(Robot,A)
      S1: At(Package,A), At(Robot,C)

--- Test C3: extra Move(A,C), no PickUp, goal Package at C
Initial: At(Package,A), At(Robot,A)
Goal   : At(Package,C)
No plan found

Invalid plan (robot never picked up package) verifies as: False
```

Prolog

```
?- can_move(a,b). true
?- can_move(a,c). false
?- valid_move(a,b). true
?- valid_move(b,c). true
?- valid_move(a,c). false
?- reduce_speed. true
?- valid_path([a,b,c]). true
?- valid_path([a,c]). false
```

Appendix C: Code

planner.py (LLM-generated, unmodified)

```
"""STRIPS-style planner with BFS for the warehouse robot (locations A, B, C).

State   : frozenset of ground propositions, e.g. ("At","Robot","A").
Action  : name, positive/negative preconditions, positive/negative effects.
Applicable(S, a): pos_pre <= S and neg_pre disjoint from S      (logic: S |= Pre(a))
Apply(S, a)      : (S - neg_eff) | pos_eff                       (effects)
Search   : breadth-first over states; goal reached when goal <= S.
Assumptions: closed-world (facts not in the set are false); the package is the only
movable object; Move is allowed only along the connected edges A-B and B-C.
"""
from collections import deque
from collections import namedtuple

Action = namedtuple("Action", "name pos_pre neg_pre pos_eff neg_eff")

LOCS = ["A", "B", "C"]
EDGES = [("A", "B"), ("B", "A"), ("B", "C"), ("C", "B")]

def at(x, l): return ("At", x, l)

def make_actions(include_pickup=True, extra_edges=()):
```

```

acts = []
for x, y in list(EDGES) + list(extra_edges):
    acts.append(Action(f"Move({x},{y})", {at("Robot", x)}, set(), {at("Robot", y)}, {at("Robot", x)}))
for l in LOCS:
    if include_pickup:
        acts.append(Action(f"PickUp(Package,{l})", {at("Robot", l)}, at("Package", l)}, set(),
                           {("Holding", "Package")}, {at("Package", l)}))
        acts.append(Action(f"Drop(Package,{l})", {at("Robot", l)}, {("Holding", "Package")}, set(),
                           {at("Package", l)}, {("Holding", "Package")}))
    return acts

def applicable(state, a):
    return a.pos_pre <= state and not (a.neg_pre & state)

def apply_action(state, a):
    return frozenset((state - a.neg_eff) | a.pos_eff)

def bfs_plan(init, goal, actions):
    """Return (plan, states) or None. plan = list of action names; states = S0..Sn."""
    init = frozenset(init)
    frontier = deque([init])
    parent = {init: None}
    while frontier:
        s = frontier.popleft()
        if set(goal) <= s:
            plan, states = [], [s]
            while parent[s] is not None:
                s, name = parent[s]
                plan.append(name)
                states.append(s)
            return plan[::-1], states[::-1]
        for a in actions:
            if applicable(s, a):
                n = apply_action(s, a)
                if n not in parent:
                    parent[n] = (s, a.name)
                    frontier.append(n)
    return None

def fmt(state):
    return ", ".join(sorted(f"{p}{'','.join(args)}" for p, *args in state))

def show(title, init, goal, actions):
    print(f"--- {title}")
    print("Initial:", fmt(init)); print("Goal   :", fmt(goal))
    res = bfs_plan(init, goal, actions)
    if res is None:
        print("No plan found")
        return None
    plan, states = res
    print("Plan:", " " -> ".join(plan))
    for i, s in enumerate(states):
        print(f"  S{i}: {fmt(s)}")
    return res

def verify(init, goal, plan, actions):
    """Independent re-execution of a plan: every precondition checked, goal checked."""
    by_name = {a.name: a for a in actions}
    s = frozenset(init)
    for name in plan:
        a = by_name[name]
        if not applicable(s, a):
            return False
        s = apply_action(s, a)
    return set(goal) <= s

I = {at("Robot", "A"), at("Package", "A")}
G = {at("Package", "C")}

if __name__ == "__main__":
    acts = make_actions()
    print("Initially applicable actions:", [a.name for a in acts if applicable(frozenset(I), a)])
    r = show("Test A: solvable warehouse problem", I, G, acts)
    print("Plan verified independently:", verify(I, G, r[0], acts))
    print()
    show("Test B: no Pickup action", I, G, make_actions(include_pickup=False))
    print()
    # Test C: extra move-only action (A->C shortcut for the robot alone); robot at C must not count as package at C
    acts_c = make_actions(extra_edges=[("A", "C")])
    r = show("Test C1: goal = Package at C, with extra Move(A,C) shortcut", I, G, acts_c)
    print("Plan verified independently:", verify(I, G, r[0], acts_c))
    print()
    show("Test C2: goal = Robot at C (package need not move)", I, {at("Robot", "C")}, acts_c)
    print()

```

```
# Test C3: robot-only moves cannot satisfy a package goal when PickUp is removed
show("Test C3: extra Move(A,C), no PickUp, goal Package at C", I, G, make_actions(include_pickup=False, extra_edges=[("A", "C")]))
# extra: a proposed invalid plan must be rejected
bad = ["Move(A,B)", "Move(B,C)", "Drop(Package,C)"]
print("\nInvalid plan (robot never picked up package) verifies as:", verify(I, G, bad, acts))
```

planner.pl

```
connected(a,b).
connected(b,a).
connected(b,c).
connected(c,b).

can_move(X,Y) :- connected(X,Y).
valid_move(X,Y) :- connected(X,Y).

% Task 8
wet_road.
slippery :- wet_road.
reduce_speed :- slippery.

% Extra: check a whole plan of moves
valid_path([_]).
valid_path([X,Y|T]) :- valid_move(X,Y), valid_path([Y|T]).
```